

Za šminkanje se grafova i to predavanje tri. Vežbe 6 20 min je transformacija sa mnogo elemenata.

$F_{max} < 1/2$

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
import scipy.fft as fft
import scipy.signal as signal
from scipy.io import wavfile

import IPython
import pickle
import time
```

/home/ignjat/oe3dos/venv/lib/python3.9/site-packages/numpy/_core/getlimits.py:545: UserWarning: Signature b'\x00\xd0\xcc\xcc\xcc\xcc\xcc\xcc\xfb\xbf\x00\x00\x00\x00\x00' for <class 'numpy.longdouble'> does not match any known type: falling back to type probe function.

This warnings indicates broken support for the dtype!

```
machar = _get_machar(dtype)
```

1.1

```
In [2]: #unosenje promenljivih
f1=1
f2=3
f3=7
fs=100

a1=1
a2=1/2
a3=3

#funkcija koja pravi nas kosinus na osnovu broja odbiraka koji mu damo(N1,N2,N3)
def cos(N):
    n = np.arange(N)
    x= a1*np.cos(2*np.pi*f1/fs*n)+a2*np.cos(2*np.pi*f2/fs*n)+a3*np.cos(2*np.pi*f3/fs*n)
    return x

#unosenje promenljivih
N1=32
N2=128
N3=1024

#pozivanje funkcije
xn_1= cos(N1)
xn_2= cos(N2)
xn_3= cos(N3)

#furijeova transformacija i normalizacija funkcije
X1 = fft.fft(xn_1)/len(xn_1)*2
X2 = fft.fft(xn_2)/len(xn_2)*2
X3 = fft.fft(xn_3)/len(xn_3)*2

#definisanje promenljivih koje ce biti potrebne za plotovanje

fs_1=np.arange(0,fs,fs/N1)
```

```
fs_2=np.arange(0,fs,fs/N2)
fs_3=np.arange(0,fs,fs/N3)

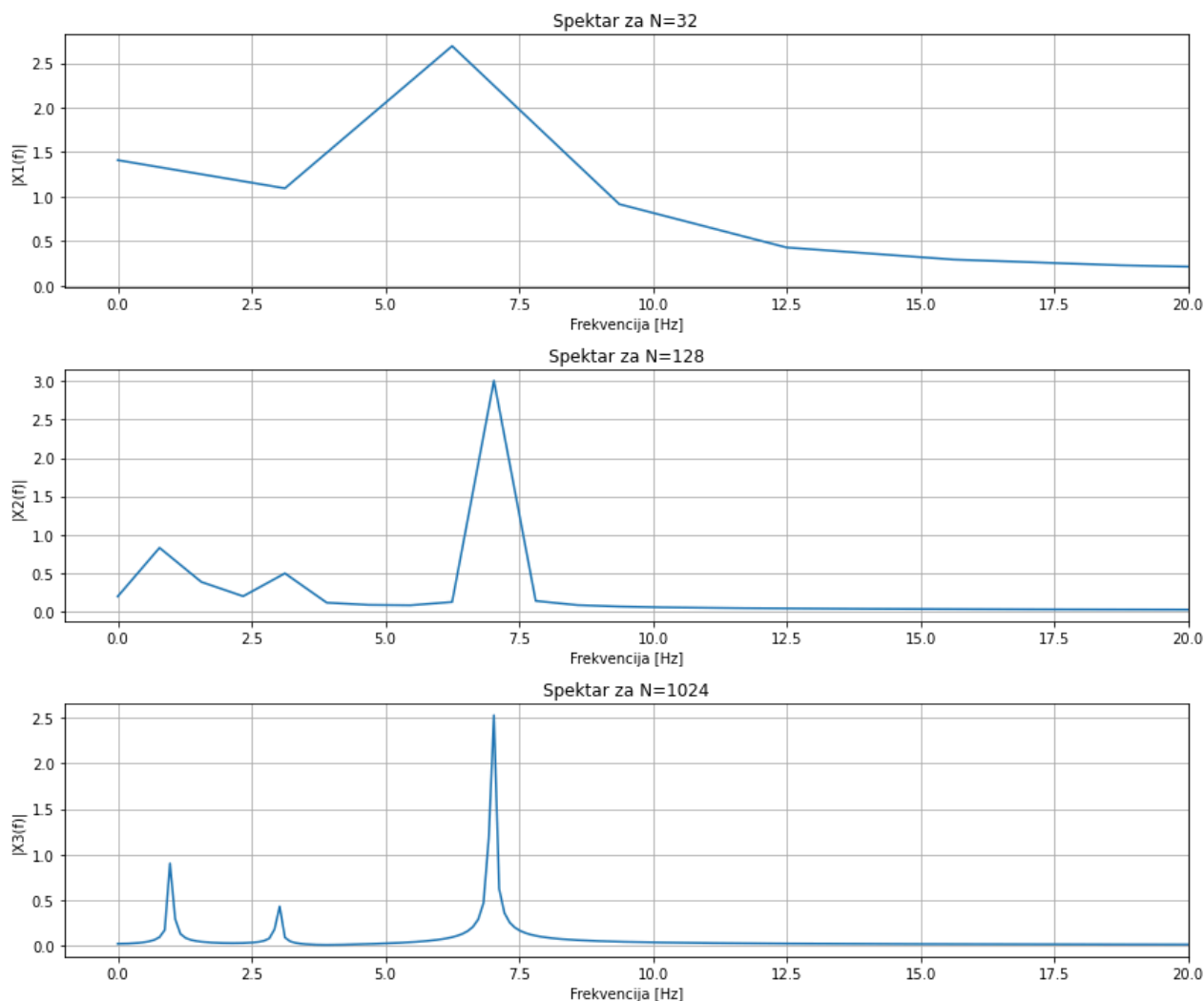
#plotovanje
fig, axes = plt.subplots(3, 1, figsize=(12, 10))

axes[0].plot(fs_1, np.abs(X1))
axes[0].set_xlim(-1, fs//5)
axes[0].set_title("Spektar za N=32")
axes[0].set_xlabel("Frekvencija [Hz]")
axes[0].set_ylabel("|X1(f)|")
axes[0].grid(True)

axes[1].plot(fs_2, np.abs(X2))
axes[1].set_xlim(-1, fs//5)
axes[1].set_title("Spektar za N=128")
axes[1].set_xlabel("Frekvencija [Hz]")
axes[1].set_ylabel("|X2(f)|")
axes[1].grid(True)

axes[2].plot(fs_3, np.abs(X3))
axes[2].set_xlim(-1, fs//5)
axes[2].set_title("Spektar za N=1024")
axes[2].set_xlabel("Frekvencija [Hz]")
axes[2].set_ylabel("|X3(f)|")
axes[2].grid(True)

plt.tight_layout()
plt.show()
```



Primećujemo da u što manjoj rezoluciji biramo (naše N je manje), to su nam rezultati manje precizni. Sa prve slike ne možemo zaključiti ništa osim da imamo potencijalni pik na frekvenciji 7 Hz, a i njega vidimo samo zbog toga što ima veću amplitudu nego ostala dva pika. Na slikama gde je $N=128$ i $N=1024$ dobijamo očekivane rezultate.

In [3]:

```
#pozivanje funkcije findpeaks koja nam je vec data
def find_peaks(X, normalize):
    X=X[:len(np.abs(X))//2+1]
    peaksPos = signal.argrelextrema(np.abs(X), np.greater)
    peaksPos = peaksPos[0]
    maxPeaksPos = np.argsort(-np.abs(X)[peaksPos])
    maxPeakIndex = peaksPos[maxPeaksPos]*normalize

    return maxPeakIndex

#faktor normalizacije
normalize1=fs/N1
normalize2=fs/N2
normalize3=fs/N3

print(find_peaks(X1, normalize1))
print(find_peaks(X2, normalize2))
print(find_peaks(X3, normalize3))
```

```
[6.25]
[7.03125 0.78125 3.125 ]
[7.03125    0.9765625  3.02734375]
```

Ovde vidimo da su pikovi postavljeni po opadajućem redosledu(na osnovu njihovih maksimuma).
Kao što je gore već navedeno, spek X2 i X3

In [5]:

```
w1 = signal.windows.triang(N1, sym=False)
w2 = signal.windows.triang(N2, sym=False)
w3 = signal.windows.triang(N3, sym=False)

xw1=xn_1*w1
xw2=xn_2*w2
xw3=xn_3*w3

XW1 = fft.fft(xw1)/len(xw1)*2
XW2 = fft.fft(xw2)/len(xw2)*2
XW3 = fft.fft(xw3)/len(xw3)*2

#definisanje promenljivih koje ce biti potrebne za plotovanje

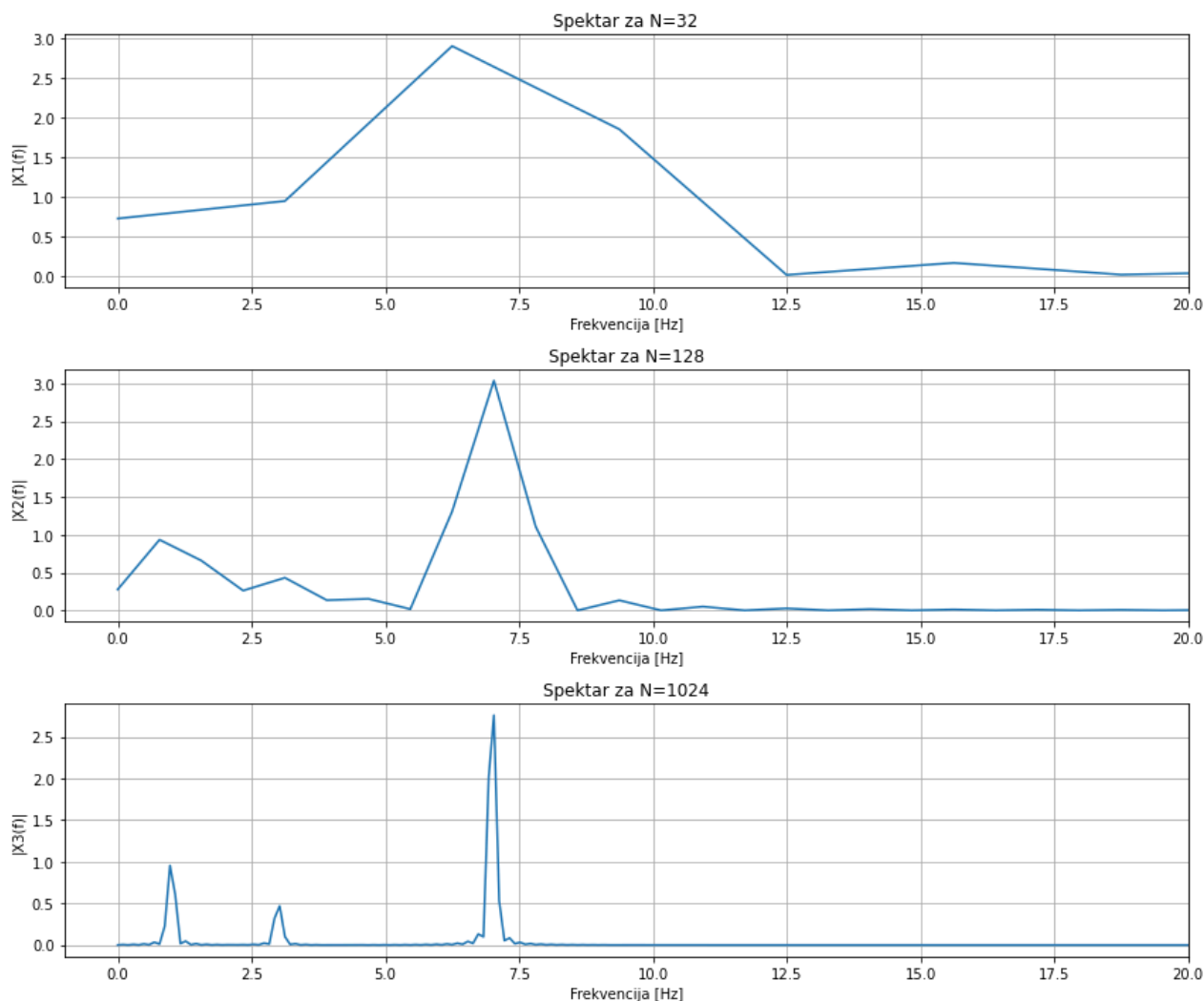
#plotovanje
fig, axes = plt.subplots(3, 1, figsize=(12, 10))

axes[0].plot(fs_1, np.abs(XW1)*2)
axes[0].set_xlim(-1, fs//5)
axes[0].set_title("Spektar za N=32")
axes[0].set_xlabel("Frekvencija [Hz]")
axes[0].set_ylabel("|X1(f)|")
axes[0].grid(True)

axes[1].plot(fs_2, np.abs(XW2)*2)
axes[1].set_xlim(-1, fs//5)
axes[1].set_title("Spektar za N=128")
axes[1].set_xlabel("Frekvencija [Hz]")
axes[1].set_ylabel("|X2(f)|")
axes[1].grid(True)

axes[2].plot(fs_3, np.abs(XW3)*2)
axes[2].set_xlim(-1, fs//5)
axes[2].set_title("Spektar za N=1024")
axes[2].set_xlabel("Frekvencija [Hz]")
axes[2].set_ylabel("|X3(f)|")
axes[2].grid(True)

plt.tight_layout()
plt.show()
```



In [6]:

```
sort_peaks1=(find_peaks(np.abs(XW1),normalize1))[0:3]
sort_peaks2=(find_peaks(np.abs(XW2),normalize2))[0:3]
sort_peaks3=(find_peaks(np.abs(XW3),normalize3))[0:3]
```

```
print(sort_peaks1)
print(sort_peaks2)
print(sort_peaks3)
#print(find_peaks(XW3,normalize3))
```

```
[ 6.25  15.625 21.875]
[7.03125 0.78125 3.125 ]
[7.03125  0.9765625  3.02734375]
```

Frekvencije su isuviše blizu jedna drugoj, tako da nam niti jedan anti-alijasing filtar ne pomaže u nalaženju njih. U situaciji $N=32$ nam nema pomoći pošto je period odabiranje isuviše mali i tu baš nemamo sreće, dok za $N=128$ i $N=1024$ vidimo da trougaoni filtar pravi neke lažne pikove. Oni nisu velikog intenziteta ali jesu lažni. Takođe trougaona funkcija nam šiti pikove pošto je ona u spektru sinc^2 . Hanova funkcija već daje bolje rezultate pošto nam jasno pokazuje gde se nalaze pikovi dok bukvalno sve ostale elemente smešta da budu jako bliski nuli.

1.2

In [7]:

```

N=625
f1=1
f2=1.32
f3=1.3
fs=12.5

a1=5
a2=1000
a3=10

n = np.arange(N)
xn= a1*np.cos(2*np.pi*f1/fs*n)+a2*np.cos(2*np.pi*f2/fs*n)+a3*np.cos(2*np.pi*f3/fs*n)

XN=fft.fft(xn)/len(xn)*2
fs_n=np.arange(0,fs,fs/N)

fig, axes = plt.subplots(3, 1, figsize=(12, 10))

maskr= np.real(XN) < 1e-10
maski= np.imag(XN) < 1e-10

XNR=np.real(XN)
XNR[maskr]=0

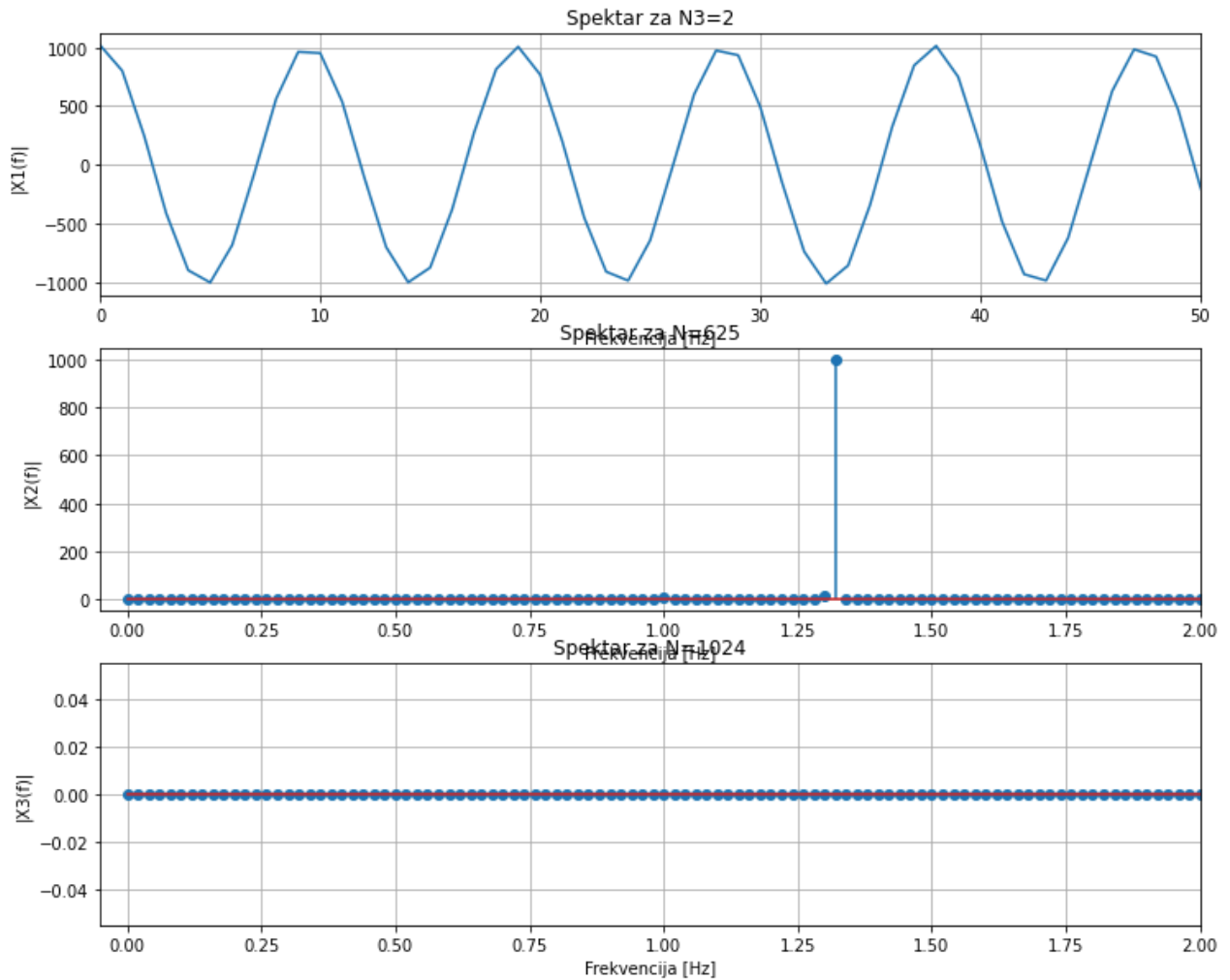
XNI=np.imag(XN)
XNI[maski]=0

#plotovanje naseg novog signala(xlim ide do N/12.5 da bi se signal jasno video)
axes[0].plot(n, xn)
axes[0].set_xlim(0, N//12.5)
axes[0].set_title("Spektar za N3=2")
axes[0].set_xlabel("Frekvencija [Hz]")
axes[0].set_ylabel("|X1(f)|")
axes[0].grid(True)

#plotovanje realnog dela
axes[1].stem(fs_n, np.real(XNR))
axes[1].set_xlim(-0.05, fs//5)
axes[1].set_title("Spektar za N=625")
axes[1].set_xlabel("Frekvencija [Hz]")
axes[1].set_ylabel("|X2(f)|")
axes[1].grid(True)

#plotovanje imaginarnog dela
axes[2].stem(fs_n, np.imag(XNI))
axes[2].set_xlim(-0.05, fs//5)
axes[2].set_title("Spektar za N=1024")
axes[2].set_xlabel("Frekvencija [Hz]")
axes[2].set_ylabel("|X3(f)|")
axes[2].grid(True)

```



Da bi dobili traženi rezultat, potrebno je da N bude toliko veliko tako da svaka da umnožak količnika frekvencije signala(f) i frekvencije odabiranja(f_s) i broja odbiraka N bude **CEO BROJ**. Time dobijamo da se frekvencije neće "iseći" u nekom trenutku i time stvoriti lažne pikove u spektru(koji bi verovatno bili na visokim frekvencijama jer se na visokim frekvencijama dešavaju nagle promene). Ako bi posmatrali naše frekvencije, videli bi da nam najveći "problem" pravi frekvencija $f_2=1,32$. Takođe vidimo i da je $f_3=1,3$, i tako možemo zaključiti da je korak koji tražimo za pravilno reprezentovanje spektra $\text{korak}=\text{abs}(f_2-f_3)=0,2$. Kada frekvenciju odabiranja podelimo tim korakom, dobijamo $f_s/\text{korak}=N=625$. Stoga u spektru jedina 3 pika koja vidimo su na realnom delu frekvenskog domena su f_1, f_2, f_3 . U imaginarnom delu su sve vrednosti jednake nuli, što nam govori da nije došlo do curenja spektra. Kada bi uzeli za N neki broj(125 na primer), naš imaginarni spektar ne bi bio ravna linija pošto bi zbog curenja spektra došlo i do promene faze.

1.3

In [8]:

```
#učitavanje signala
fsADC, x = wavfile.read('dz1_signali/singing.wav')
x = x.astype(np.float64) / 32768 # KLJUČNO

#njegovo plotovanje
fig, ax = plt.subplots(figsize = [10, 5])
plt.subplots_adjust(bottom=0.15, left = 0.15)
```

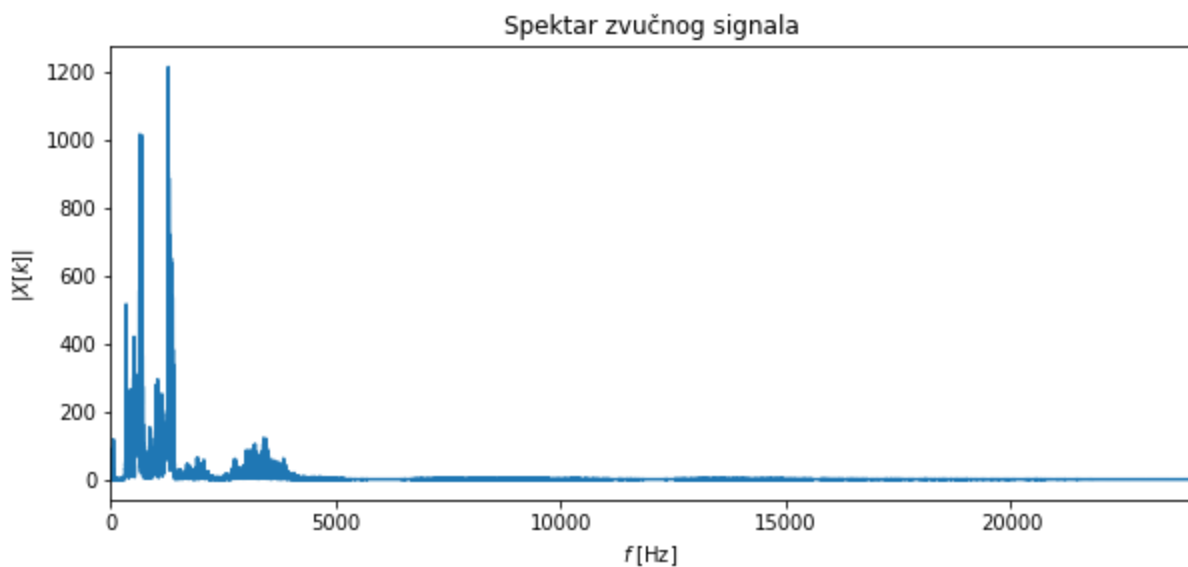
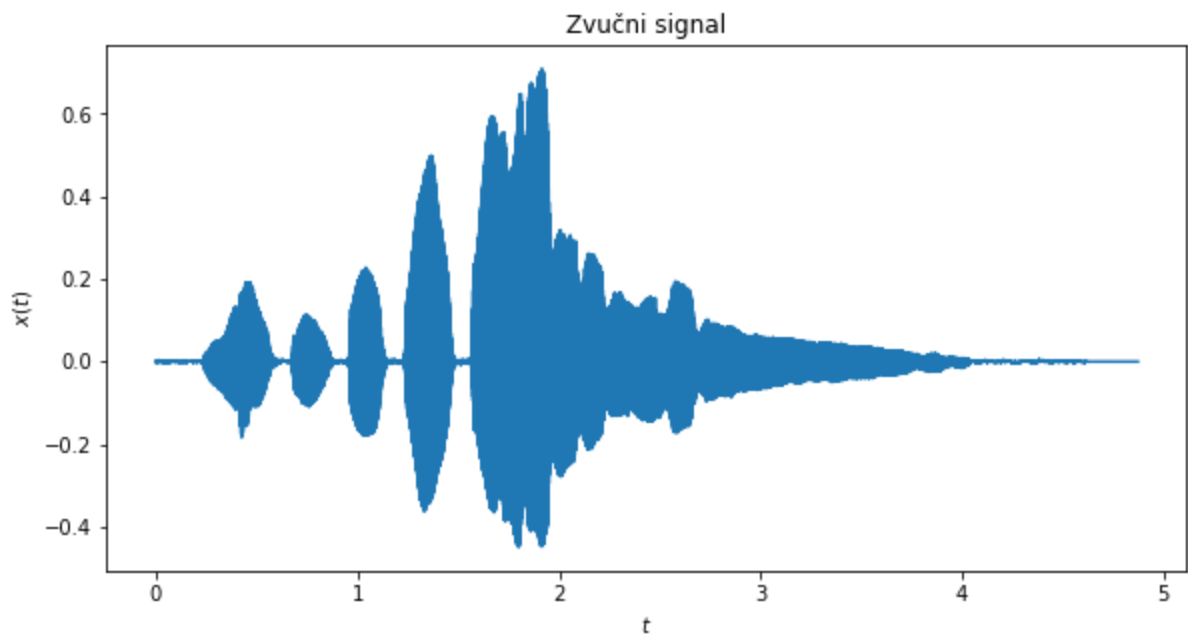
```

t = np.arange(len(x))/fsADC
plt.plot(t, x)
ax.set_title("Zvučni signal")
ax.set_xlabel(r'$t$');
ax.set_ylabel(r'$x(t)$');
plt.show()

N = len(x)
X = fft.fft(x)
k = np.arange(N)
f = fsADC*k/N

#plotovanje spektra
fig = plt.figure(figsize = [10, 5])
plt.plot(f, abs(X))
plt.subplots_adjust(bottom=0.25, left=0.15)
plt.xlim(0,np.max(f)//2)
plt.title("Spektar zvučnog signala")
plt.xlabel(r'$f$ [Hz]')
plt.ylabel(r'$|X[k]|$');

```



In [9]:

```
def obw(x,energyPct,fs):

    X=fft.fft(x)
    X2=np.abs(X)**2
    N=len(x)

    P=np.abs(X2)/N
    if N%2==0:
        #gledamo pola spektra pa dupliramo energiju
        P_freq=P[0:len(X)//2+1]
        P_freq[1:-1] *= 2
    else:
        P_freq=P[0:(len(X)+1)//2]
        P_freq[1:] *=2

    E_pct=energyPct*np.sum(P_freq)
    E_new=0

    for k in range(0,len(P_freq)):
        if E_new>=E_pct :
            kobw=k
            return kobw*fs/N
        else:
            E_new+=P_freq[k]

    return fs/2
```

In [10]:

```
print(fsADC)
print(obw(x,0.9999,fsADC))

fs_obw=(obw(x,0.9999,fsADC))
faktor=fsADC/(2*fs_obw)
print((faktor))
```

```
48000
4633.694355483532
5.179452540195775
```

Iz rezultata ispisanih u ćeliji iznad možemo videti da se signal sme odabirati nešto manjom frekvencijom od (fsADC/5). Očekivano je da ćemu izgubiti malo kvaliteta na slici, ali ne bi trebalo da dodje do aliasinga. Što možemo videti u ćeliji ispod. P.S. Koristili smo 99.99% signala i smanjili njegovu veličinu 10 puta

In [11]:

```
def audio_filt(faktor,x):
    fs_new=fsADC/faktor
    xdm=x[:,faktor]
    return fs_new,xdm

fs_new,xdm=audio_filt(5,x)
IPython.display.display(IPython.display.Audio(xdm, rate = fs_new))
IPython.display.display(IPython.display.Audio(x, rate = fsADC))
```

0:00 / 0:04

0:00 / 0:04

In [12]:

```

fs_2,xm2=audio_filt(6,x)
IPython.display.display(IPython.display.Audio(xm2, rate = fs_2))

fs_3,xm3=audio_filt(7,x)
IPython.display.display(IPython.display.Audio(xm3, rate = fs_3))

fs_4,xm4=audio_filt(8,x)
IPython.display.display(IPython.display.Audio(xm4, rate = fs_4))

fs_5,xm5=audio_filt(100,x)
IPython.display.display(IPython.display.Audio(xm5, rate = fs_5))

```

0:00 / 0:04

0:00 / 0:04

0:00 / 0:04

0:00 / 0:00

Možemo primetiti da prvu ozbiljniju degradaciju signala čujemo kada je učestanost odabiranja 8 puta manja, dok je za $l=7$ takođe dobijamo ne tako dobre rezultate koliko za $l=6$. Razlog za to je što se maksimalna frekvencija signala ($fs/2$) smanjuje i time počinjemo da gubimo ključne podatke u signalu. Na $l=8$ primećujemo dolazi do ozbiljnog alijsinga i da je odatle pa nadalje naš signal faktički neupotrebljiv (ne razaznajemo pravilno šta je na glasovnom snimku). Za vrednost $l=7$ takođe primećujemo da ima alijsing, međutim signal je u potreblijiv ($energyPct=0.99$). Ako stavimo da je $l=100$, primećujemo da u tom upsegu gotovo i nema signala.

2

In [13]:

```

def dosSpectrogram(x, fs, window, noverlap, nfft, fMaxShow):

    #definisanje promenljivih koje ce nam trebati kasnije
    Ts = 1 / fs
    N = len(x)

```

```

Nw = len(window)
Nn = Nw - noverlap
Ni = 1 + (N - Nw) // Nn

t = np.arange(Ni)*Nn*Ts

#provera da li je signal realan i na osnovu toga formiranje matrice i frekvencije
if np.isrealobj(x):

    S = np.zeros((nfft//2 + 1, Ni), dtype=complex)
    f = np.arange(nfft//2 + 1)*fs / nfft

    for i in range(Ni):
        #odabiranje segmenata i smestanje u matricu S
        segment = x[i*Nn : i*Nn + Nw] * window
        X = np.fft.fft(segment, nfft)
        S[:, i] = X[:nfft//2 + 1]
        #suzavanje matrice zbog fMaxShow
        limit = f <= fMaxShow
        S = S[limit, :]
        f = f[limit]

else:

    S = np.zeros((nfft, Ni), dtype=complex)
    f = (np.arange(0, nfft)-nfft//2)*fs/nfft

    for i in range(Ni):
        #odabiranje segmenata i smestanje u matricu S
        segment = x[i*Nn : i*Nn + Nw] * window
        X = np.fft.fftshift(np.fft.fft(segment, nfft))
        S[:, i] = X
        #suzavanje matrice zbog fMaxShow
        limit = np.abs(f) <= fMaxShow
        S = S[limit, :]
        f = f[limit]

return S, f, t

```

In [14]:

```

f0=0
fs_chirp = 8000
T = 5
N = fs_chirp * T
t_sig = np.arange(0, T, 1/fs_chirp)

n = np.arange(N)
x_chirp = signal.chirp(t_sig, f0=f0, f1=fs_chirp/2, t1=T, method='linear')

IPython.display.display(IPython.display.Audio(x_chirp, rate = fs_chirp))

```

0:00 / 0:05

In [15]:

```

Nw = 256
window = np.hanning(Nw)

```

```

noverlap = Nw // 2
nfft = 512
fMaxShow = fs_chirp / 2

# Vremenski oblik signala
fig, axes = plt.subplots(3, 1, figsize=(12, 10))

axes[0].plot(t_sig, x_chirp)
axes[0].set_xlabel('Vreme [s]')
axes[0].set_ylabel('Amplitude')
axes[0].set_xlim(0, 1/5)
axes[0].set_title('Vremenski oblik chirp signala')
axes[0].grid(True)

# Naš spektrogram
S, f, t = dosSpectrogram(x_chirp, fs_chirp, window, noverlap, nfft, fMaxShow)

im1 = axes[1].pcolormesh(t, f, 20*np.log10(np.abs(S)), shading='auto')
axes[1].set_xlabel('Vreme [s]')
axes[1].set_ylabel('Učestanost [Hz]')
axes[1].set_title('Spektrogram - dosSpectrogram')
plt.colorbar(im1, ax=axes[1])

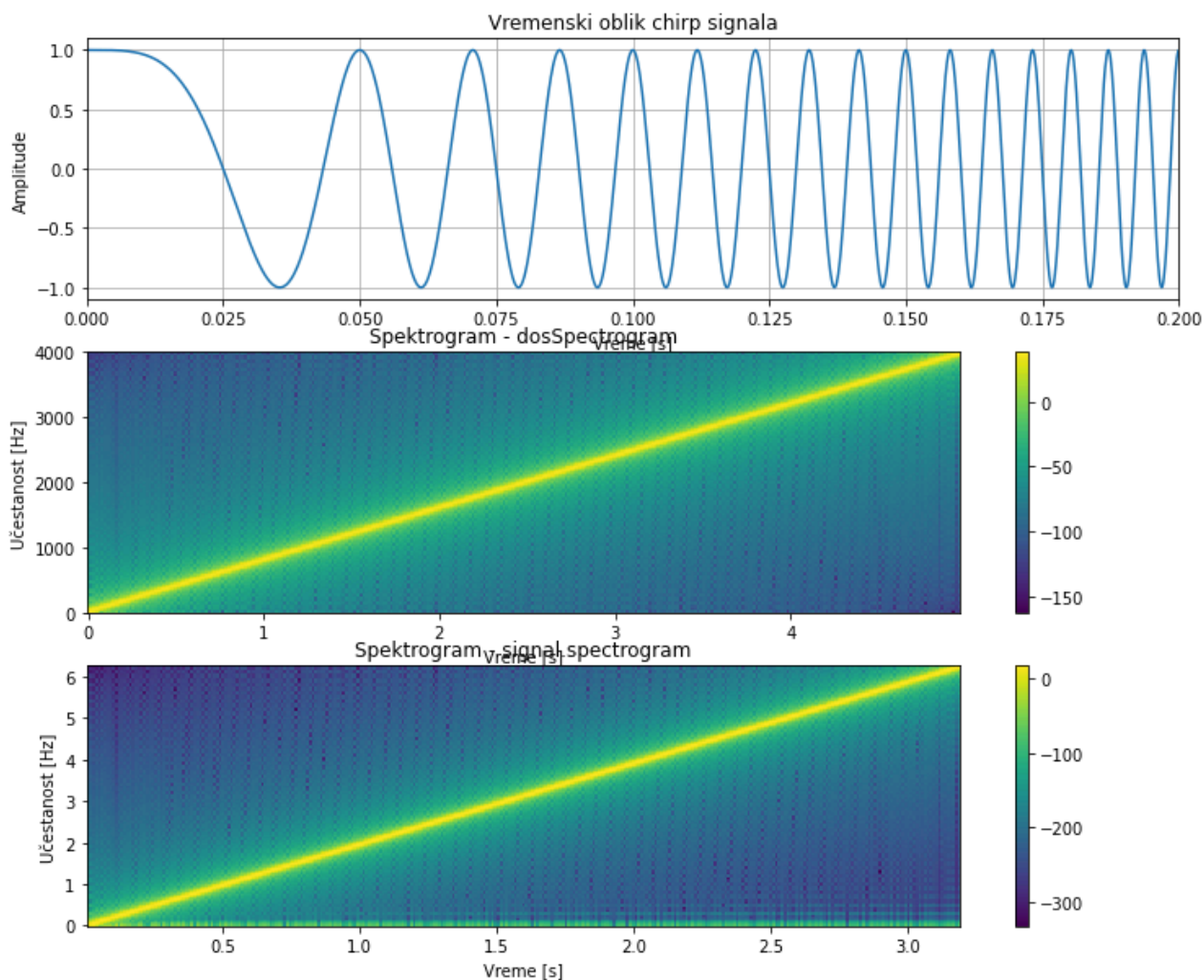
#Ugradjena funkcija spektrograma
f_sp, t_sp, Sxx = signal.spectrogram(x_chirp, fs=fs, window=window,
                                     noverlap=noverlap, nfft=nfft)

t_sp=t_sp/1000
limit = f_sp <= fMaxShow
f_sp = f_sp[limit]
Sxx = Sxx[limit, :]

im2 = axes[2].pcolormesh(t_sp, f_sp, 20*np.log10(np.abs(Sxx)), shading='nearest')
axes[2].set_xlabel('Vreme [s]')
axes[2].set_ylabel('Učestanost [Hz]')
axes[2].set_title('Spektrogram - signal.spectrogram')
plt.colorbar(im2, ax=axes[2])

```

Out[15]: <matplotlib.colorbar.Colorbar at 0x7f16695566d0>



In [16]:

```

window_lengths = [64, 256, 1024]
# 50% preklapanja
noverlap_ratio = 0.5
nfft_list = [128, 512, 2048]
fMaxShow = fs_chirp / 2

fig, axes = plt.subplots(len(window_lengths), 1, figsize=(12, 10))

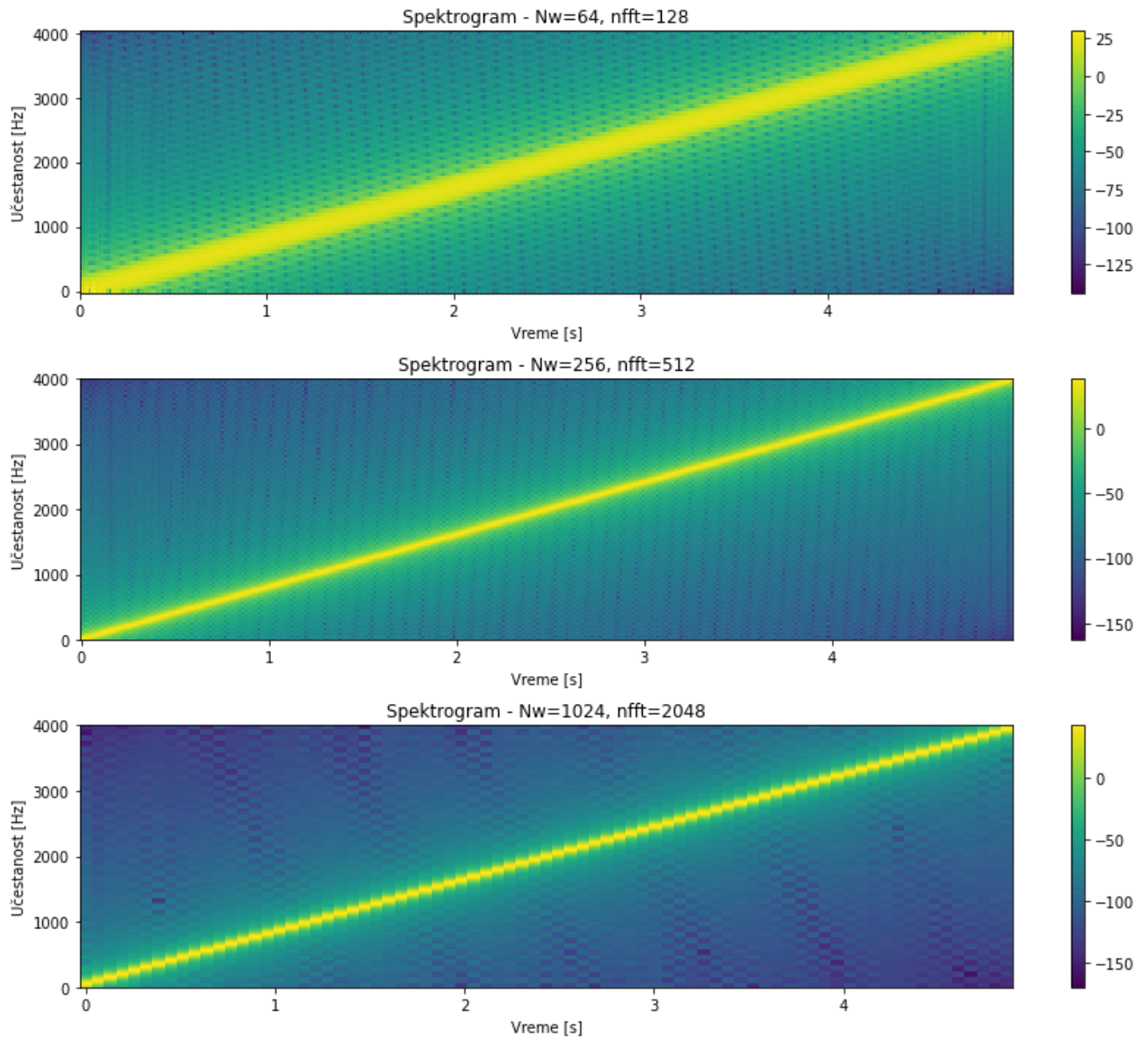
for i, Nw in enumerate(window_lengths):
    window = np.hanning(Nw)
    noverlap = int(Nw * noverlap_ratio)
    nfft = nfft_list[i]

    S, f, t = dosSpectrogram(x_chirp, fs_chirp, window, noverlap, nfft, fMaxShow)

    im = axes[i].pcolormesh(t, f, 20 * np.log10(np.abs(S) + 1e-10), shading='auto')
    axes[i].set_xlabel('Vreme [s]')
    axes[i].set_ylabel('Učestanost [Hz]')
    axes[i].set_title(f'Spektrogram - Nw={Nw}, nfft={nfft}')
    plt.colorbar(im, ax=axes[i])

plt.tight_layout()
plt.show()

```

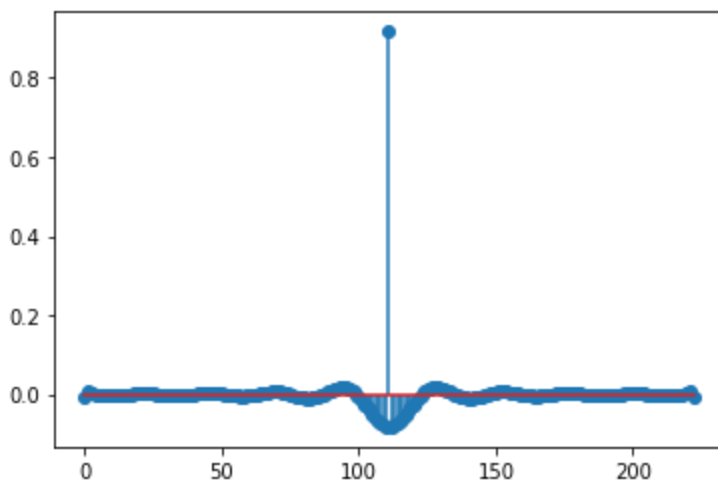


Spektar kao što možemo videti ima frekvenciju koja liearno raste, što nama i pretstavlja dokaz da naš spektorgam daje valjane rezultate. Na prvoj od ove 3 slike imamo najmanji prozor, i zbog toga je linija toliko široka i zauzima velik opseg frekvencija u jednom trenutku. Druga slika nam daje najbolje rezultate od sve 3 i time znamo da smo pogodili i širinu prozora i nfft. I na slici tri možemo videti posledice prevelikog prozora. Vremenski intervali izneđu segmenata su mnogo veliki, i kao rezultat toga ima liniju koja nam deluje isprekidano.

3.1

```
In [17]: hBirds = pickle.load(open('dz1_signali/impulse_response_birds.pkl', 'rb'))
fig = plt.figure()
plt.stem(hBirds)
```

```
Out[17]: <StemContainer object of 3 artists>
```



In [18]:

```

#3.1 Definisanje naše funkcije
def blockConvolution(x,h,blockLenght):

    #definisanje promenljivih potrebnih za kasnije
    Nx=len(x)
    Nh=len(h)
    N=Nx+Nh-1
    full_block=blockLenght+Nh-1

    #deo koji služi da proširi h da bi kasnije mogli da uradimo konvoluciju preko DFTa
    h_new=np.zeros(full_block)
    h_new[0:Nh]=h

    H=fft.fft(h_new)

    # duzine bloka u odnosu na njegovu parnost signala
    if (Nx%blockLenght==0):
        lenght = Nx//blockLenght
    else:
        lenght = Nx//blockLenght + 1

    x_final=np.zeros(lenght*blockLenght)

    #signal je podeljen na 3 dela

    for i in range(lenght):
        #i==0, prvi deo matrice se popunjava nulama
        if i==0:
            x_block=np.zeros(full_block)
            #popunjavanje matrice elementima
            x_block[Nh-1:full_block]=x[0:blockLenght]
            X_block=fft.fft(x_block)
            #kružna konvolucija
            x_new=np.real(fft.ifft(X_block*H))
        #niti jedan deo matrice se ne popunjava nulama
        elif i==(lenght-1):
            x_block=np.zeros(full_block)
            last=Nx-i*blockLenght+(Nh-1)
            x_block[0:last]=x[i*blockLenght-(Nh-1):]
            X_block=fft.fft(x_block)
            x_new=np.real(fft.ifft(X_block*H))

        #poslednji deo matrice se popunjava nulama

```

```

else:
    x_block=x[i*blockLenght-(Nh-1):(i+1)*blockLenght]
    X_block=fft.fft(x_block)
    x_new=np.real(fft.ifft(X_block*H))

    x_final[i*blockLenght:(i+1)*blockLenght]=x_new[Nh-1:full_block]
    x_final1=x_final[0:N]

return x_final1

```

In [19]:

```

#Čisto brza provera rešenja
fs_birds, x_birds = wavfile.read('dz1_signali/birds_airplane.wav')
blockL=2**9
print(len(x_birds))
print(len(hBirds))
print(np.max(blockConvolution(x_birds,hBirds,blockL)-signal.fftconvolve(x_birds,hBirds,

```

```

441000
223
5.5777604757167865e-12

```

In [20]:

```

#Pozivanje naše funkcije
my_conv=blockConvolution(x_birds,hBirds,blockL)
#Pozivanje funkcije koja je ugradejna
funct_conv=signal.fftconvolve(x_birds,hBirds,mode='full')
#Izračunavanje razlike
diff_sig=my_conv-funct_conv

Ts_birds=1/fs_birds

t=np.arange(0,len(my_conv)*Ts_birds,Ts_birds)
#plotovanje funkcije
fig, axes = plt.subplots(3, 1, figsize=(12, 10))

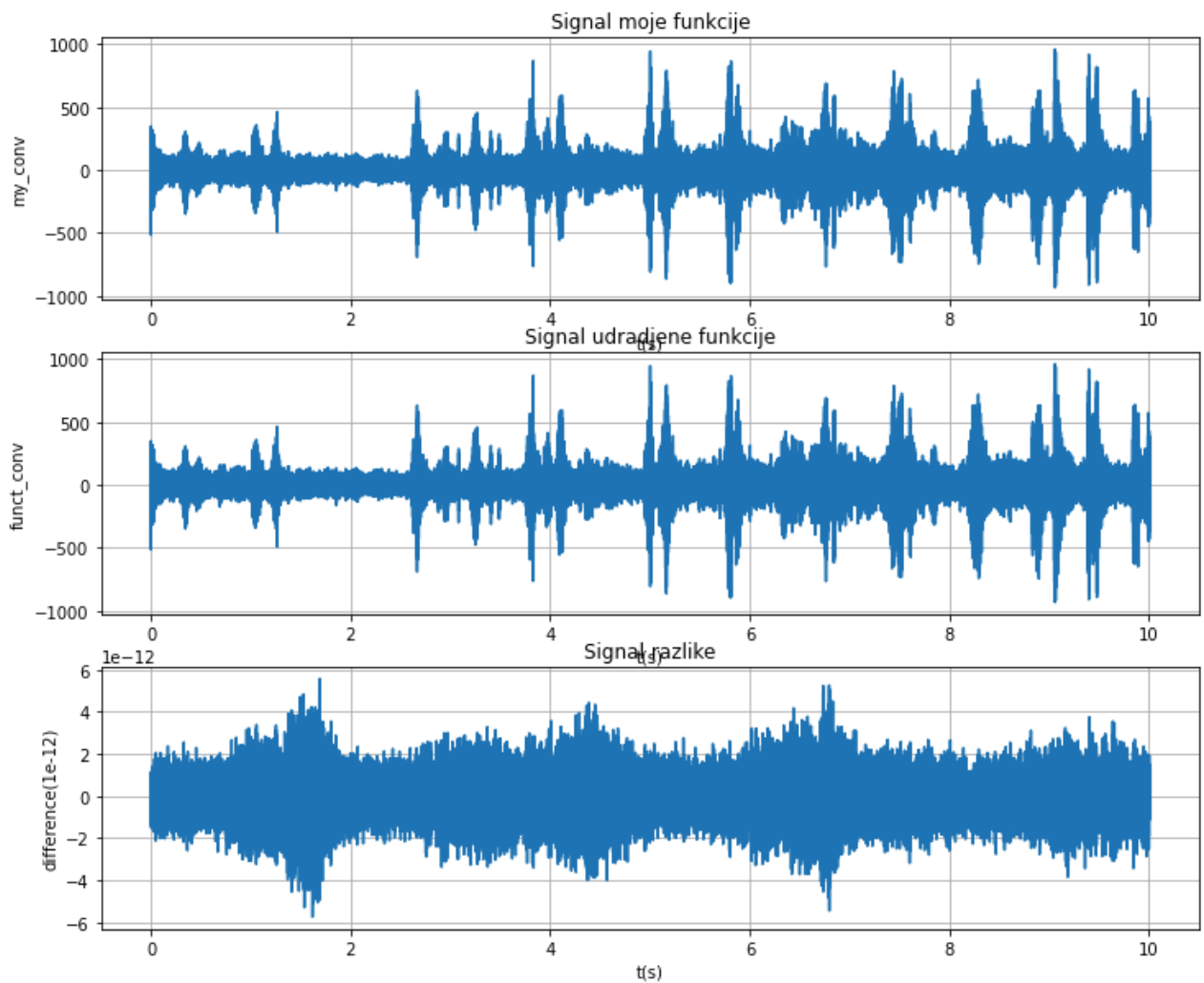
axes[0].plot(t, my_conv)
axes[0].set_title("Signal moje funkcije")
axes[0].set_xlabel("t(s)")
axes[0].set_ylabel("my_conv")
axes[0].grid(True)

axes[1].plot(t, funct_conv)
axes[1].set_title("Signal udradjene funkcije")
axes[1].set_xlabel("t(s)")
axes[1].set_ylabel("funct_conv")
axes[1].grid(True)

axes[2].plot(t, diff_sig)
axes[2].set_title("Signal razlike")
axes[2].set_xlabel("t(s)")
axes[2].set_ylabel("difference(1e-12)")
axes[2].grid(True)

plt.show()

```

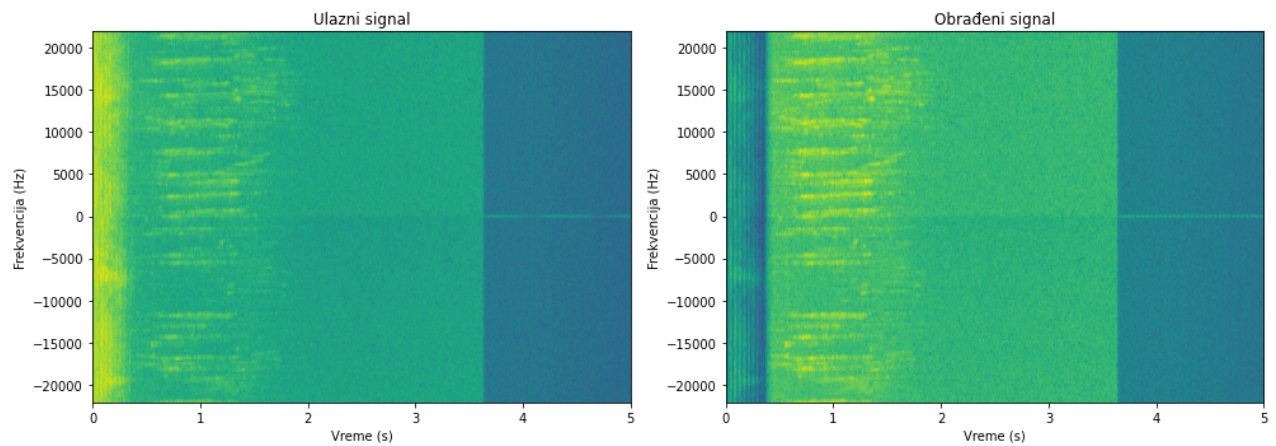


```
In [21]: fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Spektrogram ulaznog signala
ax1.specgram(fft.fft(x_birds), Fs=fs_birds)
ax1.set_title('Ulazni signal')
ax1.set_xlim(0,5)
ax1.set_xlabel('Vreme (s)')
ax1.set_ylabel('Frekvencija (Hz)')

# Spektrogram obrađenog signala
ax2.specgram(fft.fft(my_conv), Fs=fs_birds)
ax2.set_xlim(0,5)
ax2.set_title('Obrađeni signal')
ax2.set_xlabel('Vreme (s)')
ax2.set_ylabel('Frekvencija (Hz)')

plt.tight_layout()
plt.show()
```



In [22]:

```

Nx = 100000
Nh = 128
x_test = np.random.randn(Nx)
h_test = np.random.randn(Nh)

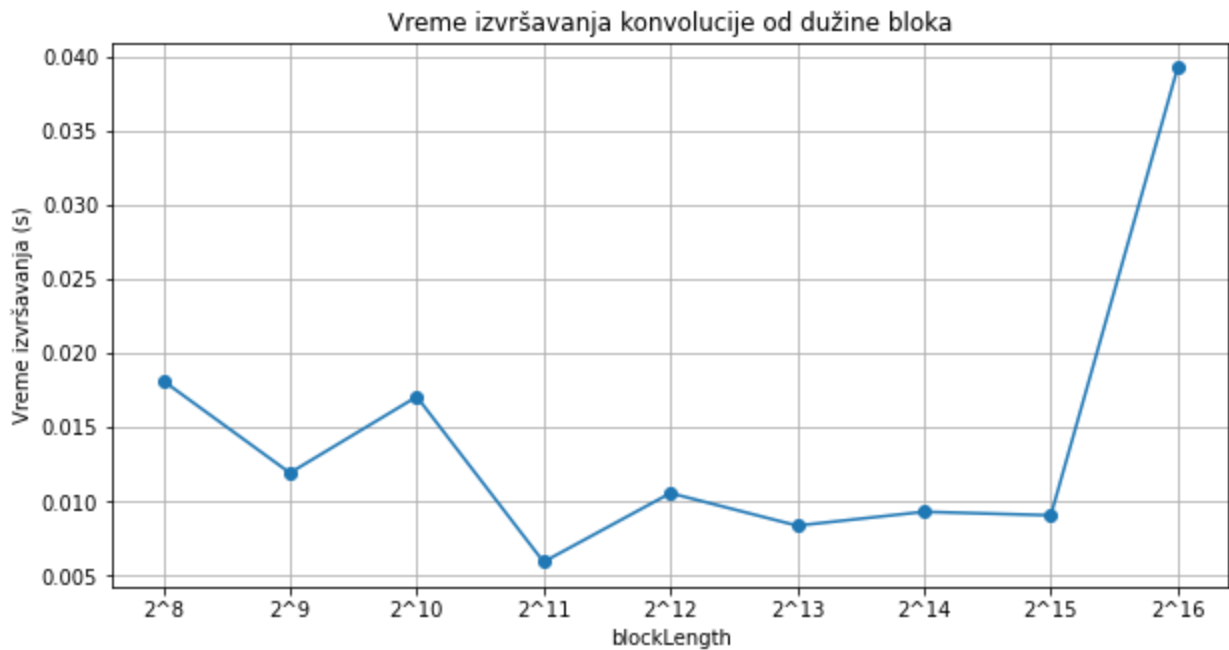
blockLengths = [2**k for k in range(8, 17)]
num_runs = 100

times = []

for blockL in blockLengths:
    start = time.time()
    for _ in range(num_runs):
        blockConvolution(x_test, h_test, blockL)
        # prosečno vreme po jednoj konvoluciji
    elapsed = (time.time() - start) / num_runs
    times.append(elapsed)

plt.figure(figsize=(10, 5))
plt.plot([f'2^{k}' for k in range(8, 17)], times, marker='o')
plt.xlabel('blockLength')
plt.ylabel('Vreme izvršavanja (s)')
plt.title('Vreme izvršavanja konvolucije od dužine bloka')
plt.grid(True)
plt.show()

```



4

In [23]:

```
#4.1.
poruka = "Ovo je jedna zanimljiva poruka koju zelim da prenesem svim dobrim ljudima na
print(len(poruka))
biti = ''.join([format(ord(c), '08b') for c in poruka])
bit_niz = np.array([int(b) for b in biti])

# ovde prenos poruke, za sada savršen
rekonstruisani_biti = bit_niz

# Biti nazad u tekst
rekonstruisana_poruka = ""
for i in range(0, len(rekonstruisani_biti), 8):
    bajt = rekonstruisani_biti[i:i+8]
    char_kod = int("".join(map(str, bajt)), 2)
    rekonstruisana_poruka += chr(char_kod)

print(f"Original: {poruka} -> Rekonstruisano: {rekonstruisana_poruka}")
```

92

Original: Ovo je jedna zanimljiva poruka koju zelim da prenesem svim dobrim ljudima na p
laneti zemlji. -> Rekonstruisano: Ovo je jedna zanimljiva poruka koju zelim da prenesem
svim dobrim ljudima na planeti zemlji.

In [24]:

```
#4.2.
def rrcFilter(beta, Nsps, filter_len_symbols):
    """
    Generiše koeficijente Root-Raised Cosine (RRC) filtra.

    Argumenti:
    beta : float - Roll-off faktor (između 0 i 1)
    Nsps : int - Broj odbiraka po simbolu (Samples Per Symbol)
    filter_len_symbols : int - Dužina filtra izražena u broju simbola

    Povratna vrednost:
```

```

h : numpy array - Niz koeficijenata impulsnog odziva filtra
"""

# Ukupan broj koeficijenata (mora biti neparan da bi filter bio simetričan)
N = filter_len_symbols * Nsps
n = np.arange(-N/2, N/2 + 1)

# Inicijalizacija niza koeficijenata
h = np.zeros(len(n))

# Skaliranje vremena (normalizovano na Nsps)
# t_over_T = n / Nsps

for i, val in enumerate(n):
    t = val / Nsps

    # Slučaj 1: Centralni odbirak (t = 0)
    if val == 0:
        h[i] = 1.0 - beta + (4 * beta / np.pi)

    # Slučaj 2: Polovi (imenilac u formuli je nula)
    # To se dešava kada je  $1 - (4 * beta * t)^2 = 0 \Rightarrow t = \pm 1/(4*beta)$ 
    elif beta != 0 and np.abs(val) == Nsps / (4 * beta):
        h[i] = (beta / np.sqrt(2)) * (
            (1 + 2/np.pi) * np.sin(np.pi / (4 * beta)) +
            (1 - 2/np.pi) * np.cos(np.pi / (4 * beta))
        )

    # Slučaj 3: Opšta formula
    else:
        numer = (np.sin(np.pi * t * (1 - beta)) +
                 4 * beta * t * np.cos(np.pi * t * (1 + beta)))
        denom = np.pi * t * (1 - (4 * beta * t)**2)
        h[i] = numer / denom

# Normalizacija energije (Veoma važno!)
# Prema Parsevalovoj teoremi, želimo da filter ima jediničnu energiju
# kako ne bi pojačavao ili slabi signal/šum.
return h / np.sqrt(np.sum(h**2))

```

In [25]:

```

Nsps=16
beta=0.3
len_symb=8
#
rrc_time=rrcFilter(beta,Nsps,len_symb)
rrc_freq=fft.fftshift(fft.fft(rrc_time)) #

N=len(rrc_time)
n=np.arange(N)-N//2

Nf = len(rrc_freq)
korak = Nsps / len(rrc_freq)

nf=np.arange(-Nsps//2,Nsps//2,korak)

rrc_convolve_time=signal.fftconvolve(rrc_time,rrc_time,mode='full')
rrc_convolve_freq=fft.fftshift(fft.fft(rrc_convolve_time))/normalize2

```

```
Nc=len(rrc_convolve_time)
nc=np.arange(Nc)-Nc//2

Nfc = len(rrc_convolve_freq)
korakc = Nsps / len(rrc_convolve_freq)

nfc=np.arange(-Nsps//2,Nsps//2,korakc)

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

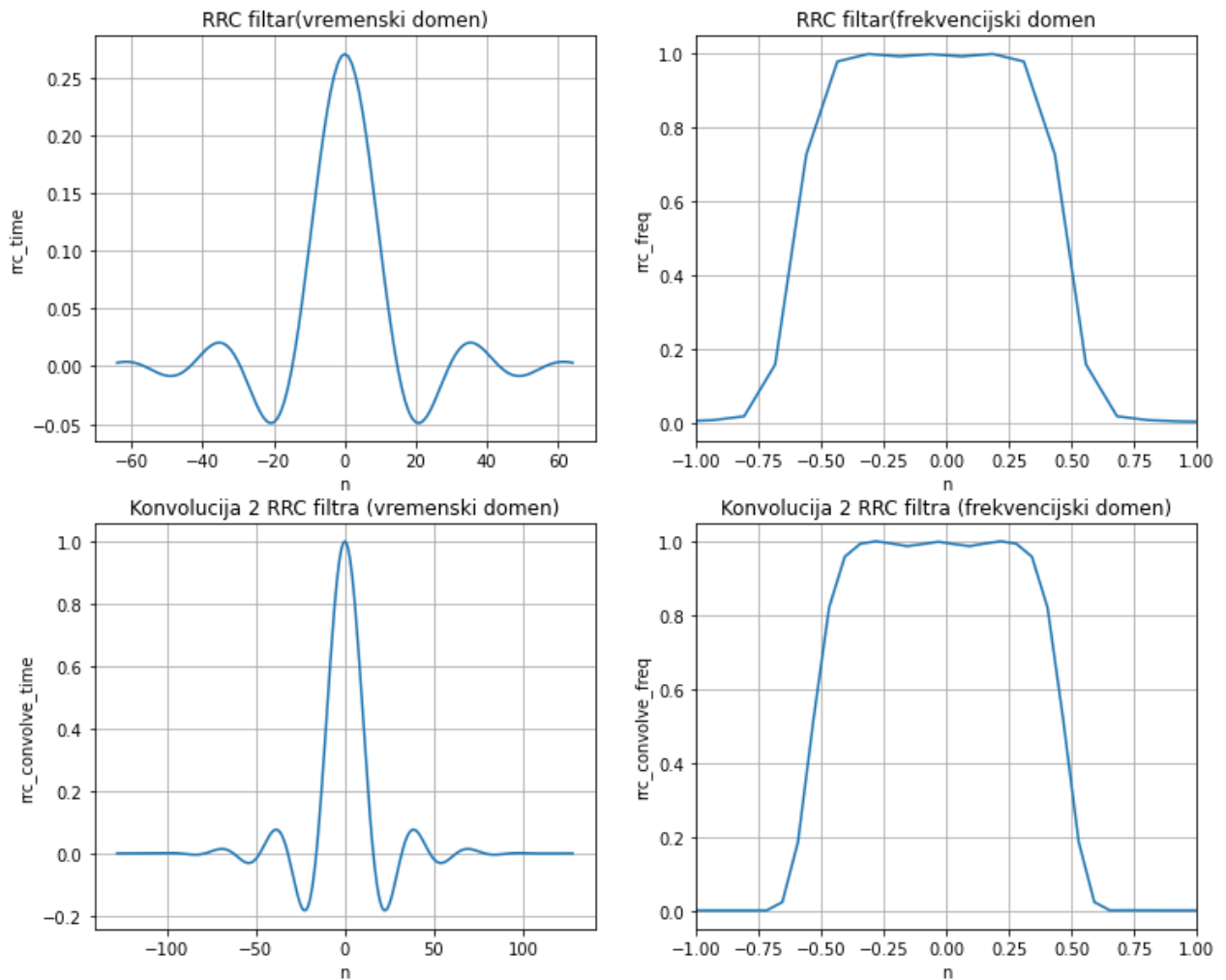
axes[0,0].plot(n, rrc_time)
axes[0,0].set_title("RRC filter (vremenski domen)")
axes[0,0].set_xlabel("n")
axes[0,0].set_ylabel("rrc_time")
axes[0,0].grid(True)

axes[0,1].plot(nf,np.abs(rrc_freq)/np.max(np.abs(rrc_freq)))
axes[0,1].set_xlim(-1,+1)
axes[0,1].set_title("RRC filter (frekvencijski domen)")
axes[0,1].set_xlabel("n")
axes[0,1].set_ylabel("rrc_freq")
axes[0,1].grid(True)

axes[1,0].plot(nc, (rrc_convolve_time))
axes[1,0].set_title("Konvolucija 2 RRC filter (vremenski domen)")
axes[1,0].set_xlabel("n")
axes[1,0].set_ylabel("rrc_convolve_time")
axes[1,0].grid(True)

axes[1,1].plot(nfc, np.abs(rrc_convolve_freq)/np.max(np.abs(rrc_convolve_freq)))
axes[1,1].set_title("Konvolucija 2 RRC filter (frekvencijski domen)")
axes[1,1].set_xlim(-1,+1)
axes[1,1].set_xlabel("n")
axes[1,1].set_ylabel("rrc_convolve_freq")
axes[1,1].grid(True)

plt.show()
```



Predajnik

In [26]:

```
#4.4
#pravljenje maske iz koje dobijamo koji su elementi 0(1) i mnozenje odgovarajucim koefi
mask= rekonstruisani_biti==0
mask1= rekonstruisani_biti==1
rekonstruisani_biti = rekonstruisani_biti.astype(complex)
rekonstruisani_biti[mask]=(np.sqrt(2)*(1+1j))
rekonstruisani_biti[mask1]=(np.sqrt(2)*(-1-1j))
rec_len=len(rekonstruisani_biti)

#4.5
#pravljenje liste koja izmedju svakog dela signala ima 15 nula
rekonstruisani_append = np.zeros(rec_len * 16, dtype=complex)
rekonstruisani_append[:16] = rekonstruisani_biti
```

In [27]:

```
#4.6
#konvolucija svih realnog i imaginarnog dela, kao i Furijeova transformacija zbira tog
convolve_signal_real=signal.convolve(np.real(rekonstruisani_append),rrc_time)
convolve_signal_imag=signal.convolve(np.imag(rekonstruisani_append),rrc_time)
convolve_full = convolve_signal_real + 1j * convolve_signal_imag
CONVOLVE_FREQ = fft.fftshift(fft.fft(convolve_full,1024))

N2=len(CONVOLVE_FREQ)
```

```
K=Nsp/N2
n2=np.arange(-Nsp//2,Nsp//2,K)
n3=128

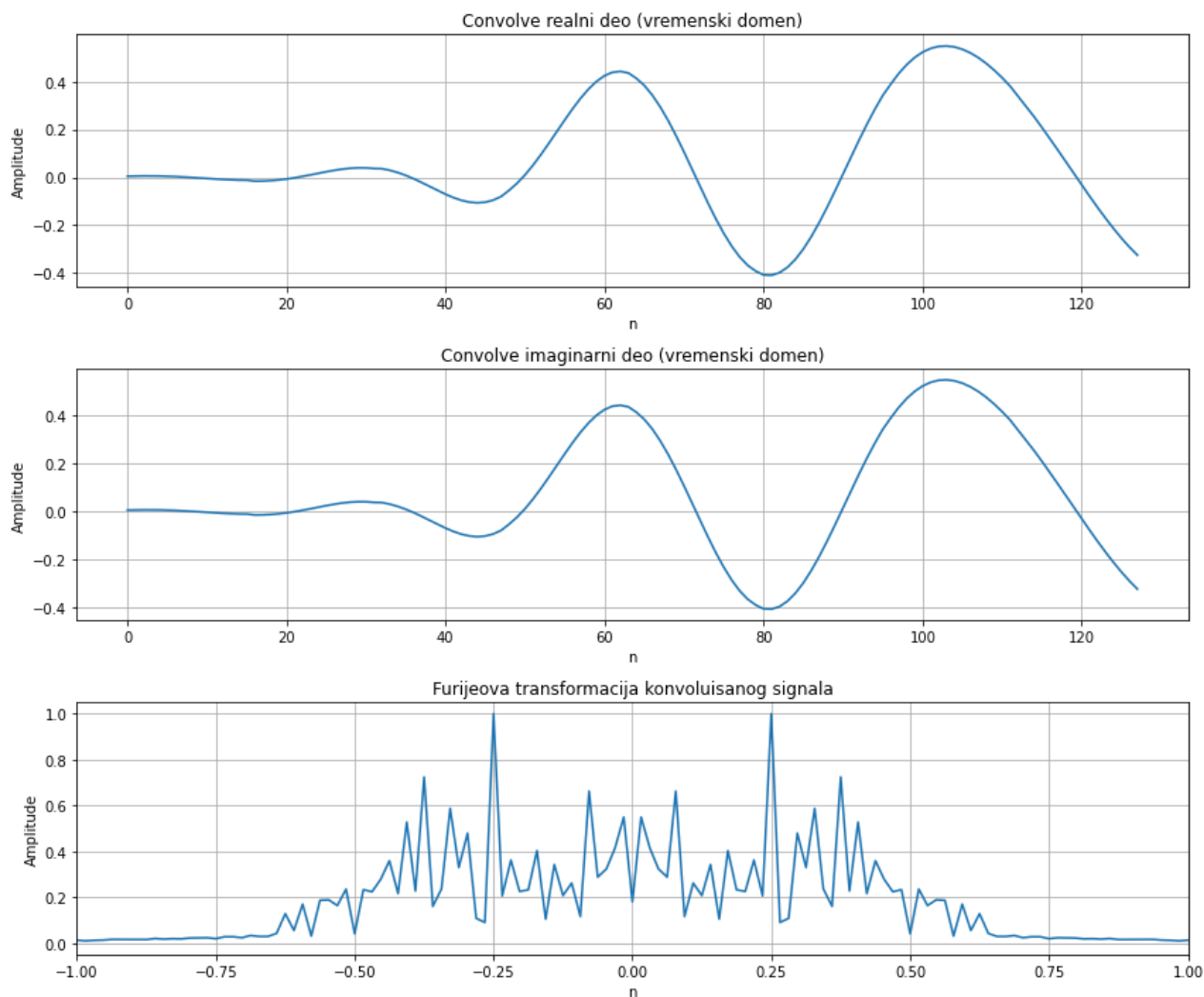
#plotovanje
fig, axes = plt.subplots(3, 1, figsize=(12, 10))

axes[0].plot(range(n3), convolve_signal_real[:n3])
axes[0].set_title("Convolve realni deo (vremenski domen)")
axes[0].set_xlabel("n")
axes[0].set_ylabel("Amplitude")
axes[0].grid(True)

axes[1].plot(range(n3), convolve_signal_imag[:n3])
axes[1].set_title("Convolve imaginarni deo (vremenski domen)")
axes[1].set_xlabel("n")
axes[1].set_ylabel("Amplitude")
axes[1].grid(True)

axes[2].plot(n2, np.abs(CONVOLVE_FREQ)/np.max(np.abs(CONVOLVE_FREQ)))
axes[2].set_xlim(-1,1)
axes[2].set_title("Furijeova transformacija konvoluisanog signala")
axes[2].set_xlabel("n")
axes[2].set_ylabel("Amplitude")
axes[2].grid(True)

plt.tight_layout()
plt.show()
```



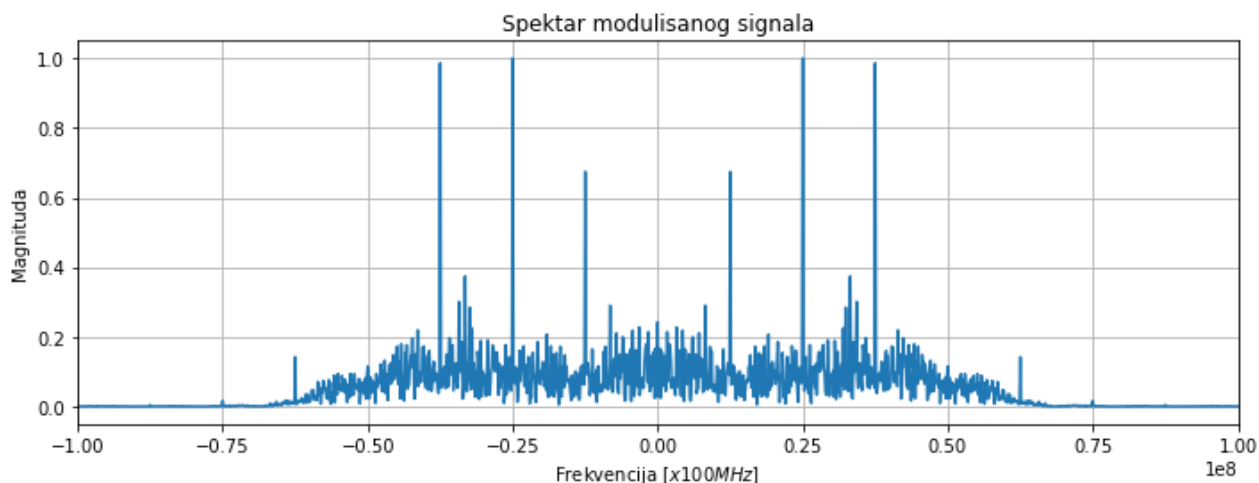
In [28]:

```
#zadatte frekvencije
FDA=1.6*(10**9)
Fn=4*(10**8)

#generisanje kompleksne sinusoide
t=np.arange(len(convolve_full))/Fn
complex_sin=np.exp(1j*2*np.pi*FDA*t)

#mnozenje i fft modulisanog signala
#normalizacija istog signala
modulate=complex_sin*convolve_full
M=fft.fftshift(fft.fft(modulate))
Nm=len(M)
normalize=FDA/Nm
freq_m=np.arange(-Nm/2,Nm/2)*normalize

#plotovanje funkcije
fig, ax = plt.subplots(figsize= (10,4))
ax.plot(freq_m,np.abs(M)/np.max(np.abs(M)))
ax.set_title('Spektar modulisanog signala')
ax.set_xlabel('Frekvencija $[x 100 MHz]$\')
ax.set_ylabel('Magnituda')
ax.set_xlim(-1*10**8,1*10**8)
plt.tight_layout()
ax.grid(True)
plt.show()
```



Kanal

```
In [29]: awgn_noise = np.random.randn(len(modulate)) + 1j * np.random.randn(len(modulate))
```

Prijemnik

```
In [31]: c=np.exp(-2j*np.pi*FDA*t)
demodulate=c*modulate #awgn_noise #(ovde se moze videti ispravan kod, ali ako obrisemo
filtered=signal.convolve(demodulate, rrc_time, method='direct')
filter_delay=filtered[128:]
reconstructed=filter_delay[:,16]

for i,num in enumerate(reconstructed):
    C1=np.sqrt((np.real(num)-np.sqrt(2))**2+(np.imag(num)-np.sqrt(2))**2)
    C2=np.sqrt((np.real(num)+np.sqrt(2))**2+(np.imag(num)+np.sqrt(2))**2)
    if C1<C2:
        reconstructed[i]=0
    else:
        reconstructed[i]=1

rekonstruisani_biti = np.real(reconstructed).astype(int)

# Biti nazad u tekst
rekonstruisana_poruka = ""
for i in range(0, len(rekonstruisani_biti), 8):
    bajt = rekonstruisani_biti[i:i+8]
    char_kod = int("".join(map(str, bajt)), 2)
    rekonstruisana_poruka += chr(char_kod)

print(f"Original: {poruka} -> Rekonstruisano: {rekonstruisana_poruka}")
```

Original: Ovo je jedna zanimljiva poruka koju zelim da prenesem svim dobrim ljudima na p
laneti zemlji. -> Rekonstruisano: ████████Mjad«██áí████0QXëíøøj61,██ïãkø%Ä@pep|£T
████Ræ²████û2\$Tº██ç;-º██/██xCpg██jgâÛý

In []: